

Milestone 1

(Capt - USMC)

Jonathan Goohs (LT - USN)

(LT - USN)

D1 - For threadTrivialWorkload.py [4 pts]

a) Based on an examination of the code available, what do you expect for relative runtime performance & output under each Python threading model?

Based off our initial examination of the code, the group expected that using the Python 3.13t environment would allow the program to execute faster due to its inherit property of running without the Python Global Interpreter Lock (GIL). This property forces only one thread to hold control of the Python interpreter at a time, which could limit runtime speed.

b) Run them once in each Python environment (3.13 & 3.13t); summarize their differences in output.

Python 3.13t: ~0.003641 seconds

Python 3.13: ~0.002218 seconds

All other outputs were the same from the script in each environment.

c) Compare your expectations to the actual behaviors, explain any differences.

Our expectations were contradictory to the actual behavior of the scripts being run. We noticed that for this program, the Python3.13 environment ran faster than the Python3.13t environment. With respect to attributing this counterintuitive result, the Python 3.13t runtime environment is built to support CPU-intensive tasks, of which this trivial workload is not.

Performance difference:

Python 3.13 was 39.1% faster than Python 3.13t.

d) Is the timing methodology adequate to make any statements about the results? Why or why not?

Yes, after running each script about 15 times each, we calculated an average percentage difference of 25% faster with Python 3.13 runtime environment.

D2 - For threadHeavyWorkload.py [4 pts]

a) Based on an examination of the code available, what do you expect for relative runtime performance & output under each Python treading model?

Based off our intial examination of the code, the group expected that using the Python 3.13t environment would allow the program to execute faster due to its inherit property of running without the Python Global Interpreter Lock (GIL). The heavy workload script performs CPU-intensive prime checking operations on large numbers, which should benefit from true parallel execution.

b) Run them, summarize their differences in output.

Python 3.13t: ~6.2 seconds

Python 3.13: ~18.5 seconds

All other outputs were the same from the script in each environment.

c) Compare your expectations to the actual behaviors, explain any differences.

Our expectations were consistent with the actual behavior of the scripts being run. Performance difference:

Python 3.13t was approximately 66% faster than Python 3.13.

With respect to attributing this result, the Python 3.13t runtime environment is built to support

CPU-intensive tasks, of which this heavy workload is. The prime checking operations can utilize true parallelism across multiple CPU cores.

d) Is the timing methodology adequate to make any statements about the results? Why or why not?

Yes, the timing methodology is adequate for this heavy workload. The longer execution times make measurement noise negligible compared to the actual runtime differences.

D3 - With respect to D1 & D2 parts d) [8 pts]

a) If there is a difference in answer between D1 and D2? Why

Yes, there is a significant difference between D1 and D2 results:

D1: Python 3.13 was 25-39% faster (trivial workload)

D2: Python 3.13t was ~66% faster (heavy workload)

This difference exists because the workload characteristics fundamentally differ in the following areas:

Trivial workload: Consists primarily of I/O operations (`print()`, `q.put()`, `os.getpid()`) that already released the GIL in standard Python. The GIL removal adds overhead without computational benefit.

Heavy workload: Involves CPU-intensive prime checking that can leverage true parallel execution across multiple cores, making the GIL removal overhead worthwhile.

b) If the answer is yes: What could be done to fix this?

Choose the appropriate Python version based on workload type. Use Python 3.13 for I/O-heavy tasks and Python 3.13t for CPU-intensive tasks.

If the answer is no: Why is it not necessary to adjust?

N/A

D4 - In a paragraph and/or a handful of bullet points, compare and contrast user and kernel threads providing an example problem for each where it is favored over the other. [4 pts]

- User threads are managed by user space libraries, entirely in user space. The operating system only views the process as a single-threaded process and can't see the other threads. With user threads, if one thread blocks then the entire process blocks and waits for that thread to complete. Since all threads run on one CPU, it can't utilize multiple cores. These are much faster as switches only require library-level access.

- Kernel threads are created and managed by the OS. Each thread is known to the OS and scheduled by the OS scheduler. Kernel threads are better positioned to take advantage of multiple cores in comparison to user threads. These threads are slower, as kernel calls are required to switch.

- An example where user threads are favored over kernel threads would be a bank ATM system handling many concurrent transactions where low overhead and fast context switching are important.

- An example where kernel threads are favored would be for AI applications used to

find new drugs
where intensive computational work benefits from parallelism across numerous cores
of intensive
power.

